MODULE 31 · DOMAIN 6 - EVALUATION AND MONITORING

Evaluating with MLflow: judges and scorers

Running an evaluation, choosing judges with and without ground truth, and writing your own scorers.

SECTION 6 of 6 EXAM WEIGHT 12% MODULE 31 of 40

admin@sthithapragna.com

Some examiners need the mark scheme. Some can mark without it. **Know which judge is which, and give each what it needs.**

How mlflow.genai.evaluate works

01
Data
Rows of inputs, and optional expectations.

02
Run
predict_fn calls the app, with tracing.

03
Score
Each scorer marks each trace.

04
Feedback
Results attach to traces, with rationales.

05
Compare
Runs side by side in MLflow.

ON THE EXAM

You can also evaluate traces you already have, such as production traces, without calling the app again.

What an evaluation row can contain

FIELD	HOLDS
`inputs`	The request, as a dictionary, passed to predict_fn
`outputs`	Optional: responses already generated, if not calling the app
`expectations: expected_response`	The ideal answer
`expectations: expected_facts`	Facts a correct answer must contain
`expectations: guidelines`	Per-row rules the answer must follow

ON THE EXAM

Expectations are the mark scheme. Judges that need ground truth read them from here.

Which judges need the mark scheme?

JUDGE	ASKS	NEEDS EXPECTATIONS?
`Correctness`	Matches expected facts or response?	Yes
`RetrievalSufficiency`	Retrieval supplied the expected facts?	Yes
`RelevanceToQuery`	Addresses the question?	No
`Safety`	Free of harmful content?	No
`RetrievalGroundedness, RetrievalRelevance`	Supported by relevant chunks?	No
`Guidelines`	Follows your written rules?	No

ON THE EXAM

Only Correctness and RetrievalSufficiency need ground truth. The rest can run on any trace.

Judges with ground truth, or without?

NEED EXPECTATIONS

Correctness, RetrievalSufficiency

- Tell you if the answer is right
- Need curated expected facts
- Evaluation sets only
- Strongest signal for accuracy

NO EXPECTATIONS NEEDED

Relevance, Safety, Guidelines

- Tell you if the answer is well-behaved
- Run on any trace, today
- Evaluation and monitoring
- Cannot confirm factual accuracy alone

THE TIEBREAKER

No expected answers yet? Start with the judges that need none, and build expected facts over time for Correctness.

An LLM judge, or a code-based scorer?

LLM JUDGE

Needs judgement

- Tone, helpfulness, policy compliance
- Guidelines judge with written rules
- Gives a rationale
- Costs a model call per trace

CODE-BASED SCORER

Can be checked exactly

- Valid JSON, length, required field
- Citation present, latency from the trace
- Deterministic and cheap
- Written with @scorer

THE TIEBREAKER

If a rule can be checked exactly in code, check it in code. Save LLM judges for things that need judgement.

Writing a scorer with @scorer

PART	DETAIL
Decorator	@scorer on a plain Python function
Arguments	Keyword-only: inputs, outputs, expectations, trace. Use any subset
Returns	bool, number, string, a Feedback with name, value and rationale, or a list
Names	Every metric needs a distinct name
Class-based	A Scorer subclass: for stateful logic, offline evaluation only

ON THE EXAM

For production monitoring, use @scorer functions, registered from a Databricks notebook. Scorer subclasses cannot be registered.

Why traces matter to scoring

- predict_fn runs are **traced**, so scorers can read every step.
- Retrieval judges read the **RETRIEVER span**, from module 10.
- Scorers that take a trace need **real traces**, not a static CSV of answers.
- Each score is attached to its trace, with the rationale, for review.
- The old Agent Evaluation @metric is now @scorer, with a simpler signature.

> ON THE EXAM
>
> When a trace-based scorer returns nothing, check that real traces exist, and that the right span types were recorded.

Turning scores into decisions

- Read **pass rates per judge**, not one blended number.
- Open the failing traces and read the **rationales**.
- Compare runs on the **same dataset**, one change at a time.
- Pair quality scores with latency and cost, as in module 12.
- A judge is a model. Check a sample of its verdicts against people, which module 32 covers.

> ON THE EXAM
>
> Scores tell you where to look. Rationales and traces tell you why.

Harbourline's evaluation suite

01
Data
300 questions, expected facts for 200.

02
Judges
Correctness where facts exist, plus Groundedness, Safety.

03
Guidelines
Never promise a payout. Always cite a clause.

04
Code
@scorer: valid JSON, citation present, under 150 words.

05
Read
Per-judge rates, failing rationales, run to run.

ON THE EXAM

Judges where judgement is needed, code where a rule is exact, and Correctness only where expected facts exist.

> A team has 150 real user questions for its rag assistant, but no expected answers or facts yet. Writing them will take weeks.
>
> It wants a quality signal from an evaluation run today.

Which judges can the team use now?

A Correctness and RetrievalSufficiency on every row

B Correctness alone, using each question as its expected answer

C RelevanceToQuery, Safety, RetrievalGroundedness and Guidelines

D None of them, since judges cannot run without expected answers

◆ PAUSE AND DECIDE

Which judges can the team use now?

A Correctness, RetrievalSufficiency

Both need expectations the team does not have.

B Question as expected answer

Nonsense ground truth gives nonsense scores.

C Relevance, Safety, Grounded, Guidelines

None needs expectations. All can run today.

D None can run

False. Most built-in judges need no ground truth.

THE SIGNAL

No expected answers yet. Use the judges that need none, and build expected facts for Correctness later.

> An assistant's answers are read by another system. Every answer must be valid JSON containing a citations list with at least one item.
>
> The team wants this checked on every evaluation run, cheaply and consistently.

How should it be checked?

A An @scorer that parses the JSON and checks the citations list

B Ask the Safety judge to review the format of each answer

C Use Correctness, with the format written as expected facts

D Rely on production monitoring to spot any malformed output

◆ PAUSE AND DECIDE

How should it be checked?

A Code-based @scorer

Exact rule, exact check: deterministic and cheap.

B Safety judge

Checks for harm, not format.

C Correctness with format facts

A model judging syntax is costly and inconsistent.

D Production monitoring

Too late: the other system has already failed.

THE SIGNAL

Valid JSON with at least one citation is an exact rule. Exact rules belong in code.

> An engineer writes a class-based Scorer subclass that keeps internal state. It works well in offline evaluation.
>
> Registering it for production monitoring fails.

What should the engineer do?

A Increase the sampling rate before registering the scorer

B Register it from a scheduled job instead of interactively

C Add expected facts so that the scorer has ground truth

D Rewrite it as an @scorer function, registered from a notebook

◆ PAUSE AND DECIDE

What should the engineer do?

A Higher sampling rate
Sampling is not why registration fails.

B Register from a job
Registration must happen from a Databricks notebook.

C Add expected facts
Live traffic has none, and it is not the problem.

D @scorer, from a notebook
The only custom form production monitoring supports.

THE SIGNAL

Class-based scorers are offline only. For monitoring, use an @scorer function, registered from a notebook.

> A bank's assistant must follow two business rules: never promise a refund, and always tell customers to contact support for account changes.
>
> The team wants each response checked against these rules.

Which judge fits best?

A Correctness, with each business rule written as an expected fact

B The Guidelines judge, with each rule written as a guideline

C RetrievalRelevance, so the rules get retrieved every time

D Safety, since breaking these rules could cause customers harm

◆ PAUSE AND DECIDE

Which judge fits best?

A Correctness

Checks facts in an answer, not behaviour rules.

B Guidelines

Built to check plain-language rules. No ground truth needed.

C RetrievalRelevance

Scores retrieved chunks, not the response's behaviour.

D Safety

Checks harmful content. These are business rules.

THE SIGNAL

Business rules in plain language, checked on every response. That is exactly what the Guidelines judge is for.

What to carry forward

- **mlflow.genai.evaluate**: data, predict_fn, scorers. Results attach to traces.
- Only **Correctness** and **RetrievalSufficiency** need expectations.
- Judgement: **Guidelines** and other judges. Exact rules: **@scorer** code.
- @scorer: keyword args inputs, outputs, expectations, trace. Returns Feedback or values.
- Scorer subclasses: offline only. Monitoring needs @scorer, registered from a notebook.

> ON THE EXAM
>
> Next: module 32 brings subject-matter experts into the loop, and aligns judges with them.